# Report Homework Assignment 3

Mursal Stella Mohammadi

October 28, 2026

## 1 Introduction

This assignments goal is to find the greatest number in a large integer array by implementing and evaluating a sequential version and a parallel version written in CUDA C/C++ and executed on an NVIDIA GPU. For the parallel version the described reduction tree logic needs to be used.

### GPU

A Graphics Processing Unit (GPU) originally, as its name suggests, was used to enhance rendering 3D graphics and images. Its computational power is also used to solve general computational problems, like searching, sorting instead of graphics. This is known as General Purpose computing on GPU (GPGPU). A GPU is used for parallelism using a SIMD (Single Instruction, Multiple Data or SIMT (Single Instruction, Multiple Thread) execution model. NVIDIA GPUs are built from multiple Streaming Multiprocessors (SMs), each containing multiple control units and a large collection of execution datapaths called Streaming Processors (SPs) or cores.

### SIMT

GPUs follow a Single Instruction, Multiple Thread (SIMT) execution model. In SIMT, multiple threads run the same instruction simultaneously across different data. To hide memory access latency, some threads may block while memory is accessed and other threads, that have already accessed the data, may proceed with execution.

### Threads, Blocks, Grids and Warps

CUDA organizes parallel execution into four connected levels

- Thread: The smallest execution unit, running one copy of the kernel code.
- Blocks: A group of threads (like 256 or 512 threads) assigned to run on a single SM. Threads in the same block can share local data and synchronize.
- Grid: The complete set of all thread blocks launched by a single kernel call.
- Warp: A group of 32 consecutive threads within a block that execute instructions together in lockstep.

### Core (SP) and Streaming Multiprocessor (SM)

An NVIDIA GPU is divided into multiple hardware processing units called Streaming Multiprocessors (SMs). Each SM contains a collection of execution cores called Streaming Processors (SPs). One of the most powerful Nvidia processor has 82 SMs, and each SM has 128 SPs for a total of 10,496 SPs. Each SM has a small block of memory that is shared among its SPs.

**GPU Global Memory, Shared Memory, Registers**

GPU memory is organized into three main layers with different speeds and access levels:

- Global Memory: The largest memory space on the GPU, accessible by all threads across all blocks, but with much higher access latency
- Shared Memory: Fast, low-latency memory shared exclusively by threads within the same block.
- Registers: Private per-thread storage and the fastest memory on the GPU.

[2]

# 2 Method

The serial baseline performs a single linear iteration with $n - 1$ comparisons, which is sequential. every comparison depends on the result of the previous one. To parallelize this, the comparisons are reorganized as a *reduction tree*. Instead of one running maximum, the array is split into many independent parts, each producing a local maximum. These local maxima are then combined pairwise until a single value remains. On an NVIDIA GPU the reduction tree is mapped onto the three hardware hierarchies that provide communication. the *warp*, the *block*, and the *grid*. Each warp of 32 threads first reduces its 32 register values to a single value in lane 0 using the CUDA `__shfl_down_sync`. For a 32 lane warp the bit mask starts at 16 and proceeds through 8, 4, 2, and 1 [2]. At offset $k$, lane $i$ receives the value of lane $i + k$ and keeps the larger of the two. After the five steps, lane 0 holds the maximum of all 32 lanes. The helper function `warpFindMax` is used for this. The mask `0xffffffff` declares that all 32 lanes participate. Threads without data carry the neutral value `INT_MIN`. A block contains $w$ warps ($w \leq 32$). After level 1, each warp holds its maximum in lane 0. Because shuffle instructions cannot cross warp boundaries, the $w$ warp maxima are exchanged through a block wide shared array `__shared__int warpMax[32]`. Lane 0 of warp $j$ writes its value to `warpMax[j]`; a `__syncthreads()` barrier guarantees that all writes are visible before any read. Warp 0 then loads the $w$ valid entries (threads $i \geq w$ load `INT_MIN`) and applies the same `warpFindMax` reduction a second time. The block maximum ends up in lane 0 of warp 0 (in thread 0 of the block). Blocks cannot communicate through shared memory. The block maximum is therefore written to a single global variable `device_max` with `atomicMax`. The atomic operation performs a read-compare-write in one step, so concurrent writes from different blocks cannot overwrite each other. Only thread 0 of each block calls `atomicMax`.

The output buffer is initialized to `INT_MIN` on the host and copied to the device before every kernel launch. `INT_MIN` is the neutral element of the maximum operation. it cannot win against any array value, so blocks that finish early cannot corrupt the result. The initialization is repeated for each of the 30 measurement runs.

The launch configuration is fixed to 1024 blocks of 256 threads (8 warps per block) in all experiments. Because work is distributed with a grid-stride loop, the result is correct for any grid size, so the configuration only affects performance. A fixed configuration keeps the comparison between the two environments free of a second variable.
[1]

# 3 Implementation

## 3.1 Serial Version

The serial version performs a single linear scan over the array and keeps a running maximum. It is implemented as a standalone C function without any CUDA dependency.

```
int findMax(int *arr, size_t arr_size) {
    int max = arr[0];
    for (size_t i = 1; i < arr_size; i++){
        int current = arr[i];
        if (max < current)
            max = current;}
    return max;}
```

### 3.2 Parallel Version

The parallel version consists of a device-side reduction kernel and the host code that allocates device memory, copies data, launches the kernel, and copies the result back. The kernel implements the three-level reduction tree described already and makes use of the grid-stride loop [2].

```
__device__ int warpFindMax(int val){
    for (int offset = 16; offset > 0; offset >>= 1){
        int neighbour = __shfl_down_sync(0xffffffff, val, offset);
        val = (val > neighbour) ? val : neighbour;}
    return val;}

__global__ void kernelFindMax(const int *array, size_t length, int *max){
    __shared__ int warpMax[32];
    size_t thread_id = blockIdx.x * blockDim.x + threadIdx.x;
    size_t stride    = gridDim.x * blockDim.x;
    int localMax = INT_MIN;
    for (size_t i = thread_id; i < length; i += stride)
        if (array[i] > localMax)
            localMax = array[i];
    localMax = warpFindMax(localMax);
    int lane   = threadIdx.x % 32;
    int warpId = threadIdx.x / 32;
    if (lane == 0)
        warpMax[warpId] = localMax;
    __syncthreads();
    if (warpId == 0){
        int val = (threadIdx.x < blockDim.x / 32)
                    ? warpMax[threadIdx.x]
                    : INT_MIN;
        val = warpFindMax(val);
        if (threadIdx.x == 0)
            atomicMax(max, val);}}
```

The parallel version separates the three cost components with three independent pairs of CUDA events. One pair surrounds the host-to-device copy of the input array, one pair surrounds the kernel launch, and one pair surrounds the device-to-host copy of the result. The total parallel time is the sum of these three components. The serial version is timed with `clock_gettime(CLOCK_MONOTONIC)` on the host.

Each experiment is repeated 30 times. The minimum and the mean of the 30 measurements are reported.

The program accepts one command-line argument that selects the array size: `SMALL` ($2^{27}$, 0.5 GiB), `MIDDLE` ($2^{28}$, 1 GiB), or `LARGE` ($2^{27} + 2^{28}$, 1.5 GiB). The output is redirected with `freopen` into a file named after the

selected size, so that the three experiments do not overwrite each other. The hostnode is also considered and made visible with the file names.Compute capabiltiy CC7.5 and CC8.6 are being taken into account as the two envirnments used are with CC7.5 and CC8.6.
the Makefile looks like following

```
main: main.cu
        nvcc -gencode arch=compute_75,code=sm_75 \
             -gencode arch=compute_86,code=sm_86 \
             -o $@ $^

run: main
        ./main $(SIZE)
```

and is being executed like following:

```
make main
```

The experiments are executed sequentially on either c6-0 or c6-4.

```
./main SMALL
./main MIDDLE
./main LARGE
```

# 4 Experimentations and results

## Hardware and System Environment

All measurements were carried out on the IFI cluster. I used the nodes c6-0 and c6-4 found through the assignements hint /share/ifi/list-cuda-hosts.sh through python3 scr.py also given through assignemnt i got the compute capability.

Table 1: Hardware and environment comparison of the two cluster nodes.

| **Category** | **c6-0** | **c6-4** |
|---|---|---|
| **CPU** | | |
| Model | Intel(R) Xeon(R) CPU E5-1620 v2 | Intel(R) Core(TM) i9-10850K |
| Clock (base) | 3.70 GHz | 3.60 GHz |
| Clock (max / boost) | 3.70 GHz | 5.20 GHz |
| System Memory | 62 GiB | 125 GiB |
| **GPU** | | |
| Model | TU106 [GeForce RTX 2060 Rev. A] | GA102 [GeForce RTX 3090] |
| Clock (base SM) | 300 MHz | 210 MHz |
| Clock (max SM / boost) | 2130 MHz | 2130 MHz |
| Clock (max memory) | 7001 MHz | 9751 MHz |
| VRAM | 6144 MiB | 24576 MiB |
| Cores | 1920 | 10496 |
| SMs | 30 | 82 |
| **Operating System** | | |
| OS | AlmaLinux 9.8 (Olive Jaguar) | AlmaLinux 9.8 (Olive Jaguar) |

**Mean**

Table 2: Mean elapsed times over 30 runs, array size 402 653 184 (1.5 GiB). Times in ms.

| | **c6-4 (RTX 3090)** | **c6-0 (RTX 2060)** |
|---|---|---|
| Serial | 465.580 | 750.859 |
| Parallel, total | 152.820 | 230.275 |
| Host to device copy | 150.652 | 223.597 |
| Kernel | 2.157 | 6.664 |
| Device to host copy | 0.011 | 0.014 |
| Parallel / serial (%) | 32.82 | 30.67 |
| Speedup | 3.047 | 3.261 |
| Efficiency (%) | 3.72 | 10.87 |

**Minimum**

Table 3: Minimum elapsed times over 30 runs, array size 402 653 184. Times in ms.

| | **c6-4 (RTX 3090)** | **c6-0 (RTX 2060)** |
|---|---|---|
| Serial | 464.499 | 750.676 |
| Parallel, total | 152.468 | 230.138 |
| Host to device copy | 150.305 | 223.494 |
| Kernel | 2.148 | 6.602 |
| Device to host copy | 0.006 | 0.010 |
| Parallel / serial (%) | 32.82 | 30.66 |
| Speedup | 3.047 | 3.262 |
| Efficiency (%) | 3.72 | 10.87 |

The prallel version needs 32,82 % (RTX 3090) or 30,67 % (RTX 2060) of the serial time. Speedup 3,047 or 3,261. The host to device copy takes about 98,6 % or 97,1 % of the parallel time. The kernel time of 3090 is shorter than 2060 (2,157 ms vs 6,664 ms). The serial time at c6-0 is longer than on c6-4. The speed up is therefor higher. Efficiency is lower (3,7 % or 10,9 %). The runs are pretty much stable. Minimum and mean therefore are not so far apart.

# 5 Discussion

On both nodes the GPU result matched the serial maximum. The major time-consuming part is the host-to-device copy. It takes 98.6 % (c6-4) and 97.1 % (c6-0) of the mean parallel time, while the kernel takes only 1.4 % and 2.9 %. The kernel alone is faster than the serial loop, but the copy limits the total speedup. The RTX 3090 node (c6-4) is faster in every parallel component (kernel 2.157 ms against 6.664 ms). The speedup is nevertheless higher on c6-0 (3.26 against 3.05), because its CPU makes the serial version slower (750.9 ms against 465.6 ms). A possible improvement could maybe using streams.

# 6 Conclusion

In this assignment, a serial and a parallel CUDA version for finding the greatest number in an integer array were implemented and compared. The parallel version uses a three-level reduction tree:. warp shuffles inside each warp, shared memory and `__syncthreads()` inside each block, and `atomicMax` across blocks. A grid-stride loop distributes the work evenly. On both nodes the result matched the serial maximum. For 402 653 184 elements, the parallel version needed 32.8 % (RTX 3090, c6-4) and 30.7 % (RTX 2060, c6-0) of the serial time, which corresponds to speedups of 3.05 and 3.26. The host-to-device copy takes 97–99 % of the parallel time and limits the speedup. The problem is therefore more the data transfer.

# References


[1] NVIDIA Corporation: *CUDA C++ Programming Guide*. NVIDIA Corporation, 2026. URL: `https://docs.nvidia.com/cuda/cuda-programming-guide/contents.html`.

[2] Peter S. Pacheco and Matthew Malensek: *An Introduction to Parallel Programming*. Second Edition. Morgan Kaufmann (imprint of Elsevier), Cambridge, MA, United States, 2022. ISBN: 978-0-12-804605-0.